Projet ANPR

Reconnaissance automatique de plaques et détection de sens interdit

Guide de révision détaillé : code Python (VS Code), nœuds Node-RED, base de données, chaîne de détection et affichage sur le dashboard

YOLO · ByteTrack · EasyOCR · SQLite · MQTT (Cedalo) · Node-RED · ESP32

Octobre 2026

Sommaire

1. Vue d'ensemble du projet 3
1.1 Objectifs . . . 3
1.2 Architecture globale . . . 3
2. Rappels de cours utiles 5
2.1 MQTT . . . 5
2.2 Vision par ordinateur . . . 5
2.3 Base de données SQLite . . . 5
3. Les topics MQTT du projet 6
4. Le code Python (VS Code) expliqué 7
4.1 Imports et configuration . . . 7
4.2 Chargement du modèle et variables globales . . . 7
4.3 parse_plate : transformer le texte OCR en matricule . . . 8
4.4 is_wrong_way : la règle « sens interdit » . . . 8
4.5 Base de données et propriétaires . . . 8
4.6 E-mail, publication et enregistrement d'une infraction . . . 9
4.7 Callbacks MQTT et gestion de la caméra . . . 9
4.8 Le thread OCR : lecture et vote . . . 9
4.9 La boucle principale . . . 10
5. De la détection à l'affichage : la chaîne complète 11
5.1 Déroulé étape par étape . . . 11
5.2 Exemple de message ANPR/infraction . . . 12
5.3 Cas de la plaque non lue (« INCONNUE ») . . . 12
6. Les nœuds Node-RED (onglet « projet_iot ») 13
6.1 Nœuds de configuration . . . 13
6.2 Capteurs (ESP32 → dashboard) . . . 13
6.3 Caméra et infractions . . . 14
6.4 Le cœur : agréger état, ANPR UI et router commandes . . . 15
7. Le dashboard « ANPR UI » : fonctionnalités 16
7.1 Logique « identifiée / lue / non lue » . . . 16
7.2 Pourquoi un ui_template ? . . . 16
8. Communication Node-RED ↔ VS Code 17
8.1 Lancer et tester l'ensemble . . . 17
8.2 Dépannage . . . 17
9. Questions probables d'examen (avec réponses) 18
10. Fiche mémo (dernière révision) 20
Chaîne en une phrase . . . 20
Chiffres à connaître . . . 20
Nœuds Node-RED à citer . . . 20
Fichiers du projet . . . 20

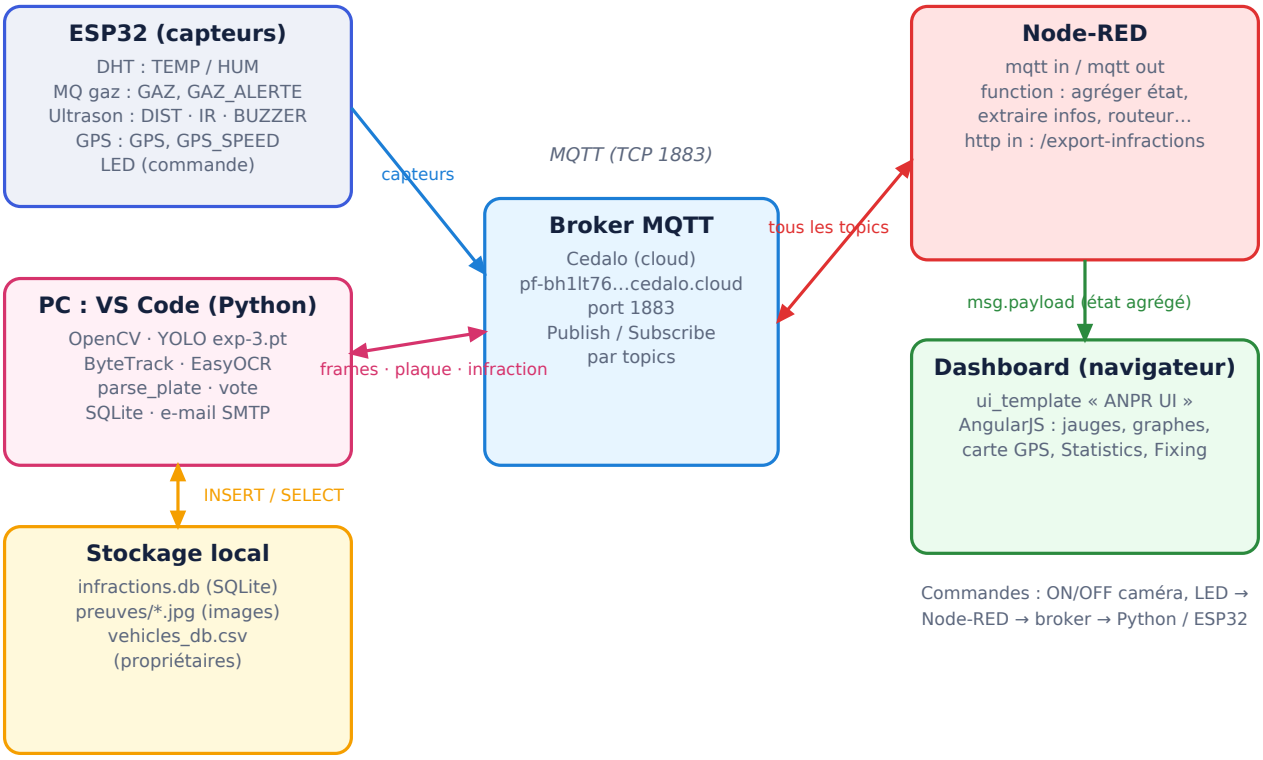
1. Vue d'ensemble du projet

Le projet combine **deux parties** qui communiquent par **MQTT** : (1) un système de vision par ordinateur écrit en Python (ouvert dans VS Code) qui détecte les véhicules/plaques, lit les matricules, repère les **sens interdits**, enregistre l'infraction en base de données et prévient le propriétaire ; (2) un **dashboard Node-RED** qui affiche en direct la caméra, les capteurs IoT d'un ESP32 et les infractions, et qui envoie des commandes (caméra, LED).

1.1 Objectifs

- **Détecter** automatiquement un véhicule qui roule à contre-sens (sens interdit) devant une caméra.
- **Identifier** le véhicule par sa plaque (OCR) et retrouver son **propriétaire** dans une base de données.
- **Enregistrer** l'infraction (SQLite + image preuve) et **notifier** le propriétaire par e-mail.
- **Superviser** en temps réel : caméra, température, humidité, gaz, distance ultrason, infrarouge, buzzer, GPS.
- **Commander** à distance : démarrer / arrêter la caméra ANPR, allumer / éteindre la LED.

1.2 Architecture globale



Composant	Rôle	Technologie
ESP32 + capteurs	Mesure température, humidité, gaz, distance, IR, GPS ; commande buzzer / LED ; publie sur MQTT.	C/C++ Arduino, MQTT (le code ESP32 n'est pas fourni : topics déduits du flow)
Script Python (VS Code)	Capture vidéo, détection, suivi, OCR, règle « sens interdit », base de données, e-mail, publication MQTT.	OpenCV, Ultralytics YOLO, EasyOCR, SQLite, paho-mqtt, smtplib
Broker MQTT Cedalo	Intermédiaire : reçoit les messages par topic et les redistribue aux abonnés.	MQTT 3.x/5, port 1883, authentification user / mot de passe
Node-RED	Reçoit tous les topics, agrège l'état, le pousse au dashboard, relaie les commandes, exporte le CSV.	Node.js, nœuds mqtt, function, template, http

Composant	Rôle	Technologie
Dashboard	Interface « ANPR UI » (AngularJS) : mode clair/sombre, jauges, graphes, carte GPS, statistiques, incidents.	HTML / CSS / AngularJS dans un ui_template
Stockage	infractions.db (table infractions), dossier preuves/ (JPEG), vehicles_db.csv (propriétaires).	SQLite, fichiers

2. Rappels de cours utiles

2.1 MQTT

- **Publish / Subscribe** : un client *publie* un message sur un **topic** (ex. `DATA/TEMP`) ; tous les clients *abonnés* à ce topic le reçoivent. Les clients ne se connaissent pas : le **broker** fait le lien.
- **Topic** : chaîne hiérarchique avec « / » (ANPR/camera/frame). **Payload** : contenu (texte, JSON, image encodée en base64).
- **QoS** : 0 = au plus une fois (rapide, perte possible, utilisé pour les images) ; 1 = au moins une fois (utilisé pour l'infraction) ; 2 = exactement une fois (abonnements Node-RED).
- **Pourquoi MQTT ici ?** Léger, adapté à l'IoT, découple le Python, l'ESP32 et Node-RED, fonctionne à travers Internet (broker cloud).

2.2 Vision par ordinateur

- **YOLO** (You Only Look Once) : réseau de neurones qui détecte des objets (boîtes + classe + confiance) en une seule passe. Ici le modèle personnalisé `models/exp-3.pt` ; ses boîtes sont recadrées puis envoyées à l'OCR, il détecte donc la plaque (déduction à partir du code).
- **ByteTrack** (`model.track(..., tracker="bytetrack.yaml", persist=True)`) : suit chaque objet d'une image à l'autre et lui donne un **identifiant de piste (tid)** stable.
- **OCR** (EasyOCR, langues arabe + anglais) : lit le texte de l'image de la plaque ; on lui fournit une liste de caractères autorisés (`OCR_ALLOWLIST`) pour limiter les erreurs.
- **Prétraitement** : niveaux de gris, agrandissement (×3 à ×8), égalisation CLAHE pour augmenter le contraste avant l'OCR.

2.3 Base de données SQLite

- Base **fichier** sans serveur, intégrée à Python (`sqlite3`). Instructions SQL : `CREATE TABLE`, `INSERT`, `UPDATE`.
- `check_same_thread=False` + un verrou (`db_lock`) permettent de l'utiliser depuis plusieurs threads sans corruption.

3. Les topics MQTT du projet

Topic	Sens	Émetteur → récepteur	Contenu	QoS
DATA/TEMP	capteur	ESP32 → Node-RED	Température °C (texte numérique)	2
DATA/HUM	capteur	ESP32 → Node-RED	Humidité %	2
DATA/GAZ	capteur	ESP32 → Node-RED	Valeur analogique 0–4095 (seuils 1365 / 2730)	2
DATA/GAZ_ALERTE	capteur	ESP32 → Node-RED	« 1 » = alerte gaz, sinon 0	2
DATA/DIST	capteur	ESP32 → Node-RED	Distance ultrason en cm (0–400)	0
DATA/IR	capteur	ESP32 → Node-RED	« 1 » = détection infrarouge	2
DATA/BUZZER	capteur	ESP32 → Node-RED	« 1 » = buzzer actif	2
DATA/GPS	capteur	ESP32 → Node-RED	« lat,lng » (si pas de fix : chaîne invalide)	0
DATA/GPS_SPEED	capteur	ESP32 → Node-RED	Vitesse	0
LED/CONTROL	commande	Node-RED → ESP32	« 1 » = ON, « 2 » = OFF	–
ANPR/camera/control	commande	Node-RED → Python	« ON » démarre, « OFF » arrête la caméra	–
ANPR/camera/frame	données	Python → Node-RED	Image JPEG 480×360 qualité 50, encodée base64 (~8 images/s)	0
ANPR/vehicle/detection	données	Python → (abonné éventuel)	JSON : horodatage + liste {class, confidence}	0
ANPR/vehicle/plate	données	Python → (abonné éventuel)	Dernier matricule lu (texte)	0
ANPR/infraction	données	Python → Node-RED	JSON de l'infraction + propriétaire	1 (pub) / 2 (sub)

À retenir : Les topics `ANPR/vehicle/detection` et `ANPR/vehicle/plate` sont publiés par Python mais aucun nœud « mqtt in » du flow ne s'y abonne : ils servent à d'autres clients ou au débogage. Pour les besoins de l'examen, le flux utile est : frame, infraction, control.

4. Le code Python (VS Code) expliqué

Le script est un programme unique avec **deux threads** : le **thread principal** (boucle vidéo : détection, suivi, règle sens interdit, envoi des images) et le **thread OCR** (`ocr_worker`), plus le thread réseau MQTT (`loop_start()`) et un thread par e-mail envoyé. L'OCR est lent : le séparer évite de bloquer la vidéo.

4.1 Imports et configuration

Bloc	Contenu / rôle
Imports	cv2 (OpenCV : caméra, images), paho.mqtt (client MQTT), YOLO (ultralytics), easyocr, sqlite3, threading, csv, re (expressions régulières), collections (Counter pour le vote, deque pour l'historique), smtplib/email (e-mail).
BASE_DIR	Dossier du script : tous les chemins (base, preuves, CSV, modèle) en sont déduits.
BROKER / PORT / USERNAME / PASSWORD	Connexion au broker Cedalo (host cloud, port 1883, identifiants).
TOPIC_*	Noms des 5 topics (frame, control, detection, plate, infraction).
SOURCE = 0, SHOW_WINDOW	SOURCE : 0 = webcam du PC (un chemin de fichier = vidéo). SHOW_WINDOW : fenêtre OpenCV locale.
SEND_FPS=8, SEND_SIZE=(480,360), JPEG_QUALITY=50	Limite le débit vers le broker : 8 images/s, réduites et compressées.
OCR_MIN_CONF=0.30	Ignore les fragments de texte dont la confiance EasyOCR est inférieure à 30 %.
VOTE_WINDOW=7, VOTE_MIN=2	Vote majoritaire : on garde les 7 dernières lectures d'une piste ; il faut 2 lectures identiques pour valider.
OCR_INTERVAL=0.7	Pause (s) avant chaque OCR pour économiser le CPU.
OCR_ALLOWLIST	Caractères autorisés : A–Z, 0–9 et le mot arabe « تونس » (Tunisie).
INFRACTION_TYPE = "SENS_INTERDIT", WRONG_WAY = "approaching"	Type enregistré ; sens interdit = véhicule qui **s'approche** de la caméra. Autres choix possibles : receding, left, right, up, down.
HIST_LEN=20, MIN_TRACK_FRAMES=10	Historique de 20 positions par piste ; au moins 10 images avant d'évaluer.
AREA_CHANGE=0.25, MOVE_PX=60	Seuil de variation de surface (+25 %) pour approaching/receding ; déplacement minimal en pixels pour left/right/up/down.
CONFIRM_EVALS=8	Il faut 8 évaluations « mauvais sens » (cumulées) avant de suspecter l'infraction.
PLATE_WAIT=10, COOLDOWN=60	Attente max de 10 s pour lire la plaque ; 60 s sans nouvelle infraction pour la même plaque.
DB_PATH, EVIDENCE_DIR, USERS_CSV, MODEL_PATH	infractions.db ; dossier preuves/ (créé s'il n'existe pas) ; vehicles_db.csv ; models/exp-3.pt.

Sécurité : Les identifiants du broker et le mot de passe applicatif Gmail sont écrits en clair dans le code. C'est une faiblesse de sécurité : on devrait les placer dans des variables d'environnement ou un fichier `.env` non versionné, et révoquer ceux déjà exposés. Ils ne sont volontairement pas reproduits dans ce document.

4.2 Chargement du modèle et variables globales

```
model  = YOLO(MODEL_PATH)                      # détecteur personnalisé
reader = easyocr.Reader(['ar', 'en'], gpu=False) # OCR arabe + anglais, sur CPU

camera_active = False   # la caméra ne démarre que sur commande « ON »
cap = None              # objet cv2.VideoCapture
ocr_crops   = {}        # tid -> dernier recadrage de plaque à lire (protégé par ocr_lock)
votes_by_tid = {}       # tid -> deque des 7 dernières lectures
plate_by_tid = {}       # tid -> plaque validée (après vote)
tracks      = {}        # tid -> {hist, wrong, pending, reported, last}
last_infraction = {}    # (plaque, type) -> instant de la dernière infraction (cooldown)
late_update = {}        # tid -> (id_infraction, horodatage) pour plaque « INCONNUE » à corriger
```

Deux verrous (**Lock**) évitent les accès simultanés : `ocr_lock` pour le dictionnaire de recadrages, `db_lock` pour les écritures SQLite.

4.3 parse_plate : transformer le texte OCR en matricule

L'OCR renvoie des morceaux de texte désordonnés. `parse_plate(tokens)` les assemble et reconnaît trois formats :

Cas	Règle	Résultat
Plaque tunisienne avec le mot arabe (تونس)	Séparer le texte autour du mot arabe : chiffres à gauche et à droite. Le format tunisien s'écrit « série TUN numéro ».	« 123 TUN 4567 » via _format_tn(left, right) ; 1–3 chiffres + 1–4 chiffres
Plaque française / européenne (7 caractères)	Corriger les confusions : positions 0,1,5,6 doivent être des lettres (TO_LETTER : 0→O, 1→I, 5→S…), positions 2,3,4 des chiffres (TO_DIGIT : O→0, I→1, S→5…). Valider par l'expression FR_RE = ^[A-Z]{2}\d{3}[A-Z]{2}$.	« AB-123-CD »
Plaque tunisienne latine / RS	Au moins 2 groupes de chiffres et ≤ 3 lettres : milieu « RS » si présent, sinon « TUN ».	« 123 RS 4567 » ou « 123 TUN 4567 »
Chiffres seuls (5 à 7)	Les 4 derniers chiffres = numéro, le reste = série.	« 12 TUN 3456 »
Autre	Pas de format reconnu.	None (lecture ignorée)

Les dictionnaires TO_LETTER et TO_DIGIT corrigent les erreurs classiques de l'OCR (O/0, I/1, S/5, B/8, G/6, Z/2, A/4).

4.4 is_wrong_way : la règle « sens interdit »

```
def is_wrong_way(hist):
    x0, y0, a0 = hist[0]      # première position (centre x, centre y, aire de la boîte)
    x1, y1, a1 = hist[-1]     # dernière position
    ratio = a1 / a0 if a0 > 0 else 1.0
    if WRONG_WAY == "approaching":
        return ratio >= 1 + AREA_CHANGE   # la boîte a grossi de 25 % : le véhicule s'approche
    ...
```

Principe : avec une caméra fixe, un objet qui **grossit** dans l'image se rapproche de l'objectif. Si l'on filme une route dont le sens autorisé éloigne les voitures, une voiture qui s'approche roule à contre-sens. On compare la **première** et la **dernière** des 20 positions mémorisées (deque de longueur HIST_LEN).

4.5 Base de données et propriétaires

```
def init_db():
    conn = sqlite3.connect(DB_PATH, check_same_thread=False)
    conn.execute("""CREATE TABLE IF NOT EXISTS infractions (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        horodatage TEXT NOT NULL, plaque TEXT NOT NULL, type TEXT NOT NULL,
        piste_id INTEGER, image TEXT, statut TEXT NOT NULL DEFAULT 'EN_ATTENTE')""")
    conn.execute("CREATE INDEX IF NOT EXISTS idx_plaque ON infractions(plaque)")
    conn.commit(); return conn
```

- **init_db()** crée la table au premier lancement et un **index** sur la plaque (recherche rapide).
- **norm_plate(p)** : met en majuscules et supprime tout sauf A–Z et 0–9 → « 123 TUN 4567 » et « 123-tun-4567 » donnent la même clé « 123TUN4567 ».

- **load_users()** lit `vehicles_db.csv` (colonnes *matricule, nom, prenom, email, telephone, carte_grise*) et construit un dictionnaire {matricule normalisé : ligne}.
- **owner_info(plate)** : cherche la plaque dans ce dictionnaire et renvoie {nom, prenom, email, telephone, carte_grise}, ou {} si le véhicule est inconnu.

> **À retenir :** Deux « bases » sont donc utilisées : **vehicles_db.csv** (registre des propriétaires, en lecture) et **infractions.db** SQLite (journal des infractions, en écriture).

4.6 E-mail, publication et enregistrement d'une infraction

- **send_infraction_email(owner, plate, timestamp)** : si le propriétaire a un e-mail, construit un message (MIMEMultipart) et l'envoie via `smtp.gmail.com:587` avec STARTTLS ; exécuté dans un **thread séparé** pour ne pas bloquer la vidéo.
- **publish_infraction(inf_id, stamp, plate, update)** : publie sur `ANPR/infraction` (QoS 1) un JSON avec *id, horodatage, plaque, type, statut, proprietaire, mise_a_jour*.
- **register_infraction(tid, plate, frame)** : voir ci-dessous.

```
def register_infraction(tid, plate, frame_with_boxes):
    now = time.time(); key = (plate, INFRACTION_TYPE)
    if plate != "INCONNUE" and now - last_infraction.get(key, 0) < COOLDOWN:
        return None                       # 1) anti-doublon : même plaque < 60 s
    last_infraction[key] = now
    cv2.imwrite(image_path, frame_with_boxes)          # 2) image preuve dans preuves/
    with db_lock:                                       # 3) INSERT SQLite (verrouillé)
        cur = db.execute("INSERT INTO infractions (...) VALUES (?,?,?,?,?)", (...))
        db.commit(); inf_id = cur.lastrowid
    publish_infraction(inf_id, stamp, plate)            # 4) MQTT -> Node-RED
    threading.Thread(target=send_infraction_email, ...).start()   # 5) e-mail
    if plate == "INCONNUE": late_update[tid] = (inf_id, stamp)    # 6) correction ultérieure
```

Les **paramètres « ? »** de la requête SQL (requête paramétrée) empêchent l'injection SQL. `cur.lastrowid` donne l'identifiant auto-incrémenté de la ligne insérée.

4.7 Callbacks MQTT et gestion de la caméra

```
def on_connect(client, userdata, flags, reason_code, properties):
    if reason_code == 0: client.subscribe(TOPIC_CAMERA_CONTROL)   # écoute les commandes

def on_message(client, userdata, msg):
    payload = msg.payload.decode().strip().strip('"').upper()
    if payload == "ON":  cap = open_source(); reset_tracking(); camera_active = True
    elif payload == "OFF": camera_active = False; cap.release(); cap = None; reset_tracking()
```

- Python n'**est abonné qu'à un seul topic** : `ANPR/camera/control`. Il reçoit « ON » ou « OFF » depuis le dashboard.
- **open_source()** ouvre la webcam avec `cv2.CAP_DSHOW` (Windows) et un buffer de 1 image (faible latence) ; pour une vidéo, ouvre le fichier.
- **reset_tracking()** vide pistes, votes, plaques et recadrages pour repartir à zéro.

4.8 Le thread OCR : lecture et vote

```
while running:
    with ocr_lock:                          # prend un recadrage en attente
        todo = [k for k in ocr_crops if k not in plate_by_tid]
        urgent = [k for k in todo if tracks.get(k, {}).get("pending")]
        tid = (urgent or todo)[0]; crop = ocr_crops.pop(tid)     # priorité aux suspects
    time.sleep(OCR_INTERVAL)
    results = reader.readtext(preprocess(crop), allowlist=OCR_ALLOWLIST, detail=1)
    results.sort(key=lambda r: r[0][0][0])                       # tri gauche -> droite
    parts = [t for _, t, c in results if c >= OCR_MIN_CONF and t.strip()]
    text = parse_plate(parts)
    votes = votes_by_tid.setdefault(tid, deque(maxlen=VOTE_WINDOW)); votes.append(text)
    best, count = Counter(votes).most_common(1)[0]
    if count >= VOTE_MIN: plate_by_tid[tid] = best               # plaque validée
```

- Les pistes **suspectes** (pending) passent en priorité.

- **Vote majoritaire** : une seule lecture peut être fausse ; on exige 2 lectures identiques parmi les 7 dernières (Counter.most_common).
- Une fois validée : publication de la plaque sur ANPR/vehicle/plate. Si une infraction avait été enregistrée avec « INCONNUE » pour cette piste (late_update), on fait un **UPDATE SQL** de la plaque, on republie l'infraction avec mise_a_jour = true et on envoie l'e-mail.

4.9 La boucle principale

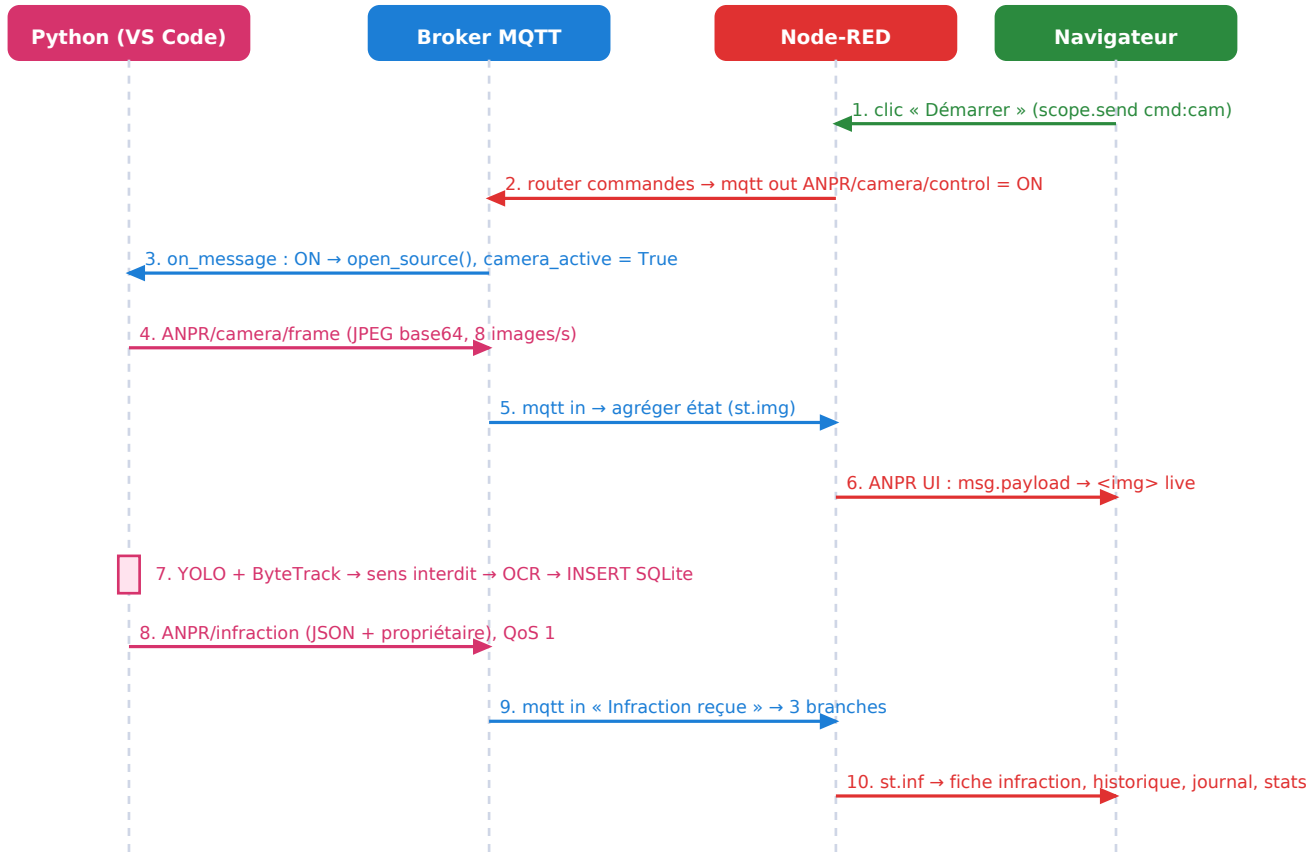
```
while True:
    if camera_active and cap and cap.isOpened():
        ret, frame = cap.read()
        results = model.track(frame, conf=0.5, persist=True, tracker="bytetrack.yaml", verbose=False)
        boxes = results[0].boxes; annotated = results[0].plot()
        ids = boxes.id.int().tolist() if boxes.id is not None else []
        for box, tid in zip(boxes, ids):
            (cx, cy, area) -> tr["hist"].append(...)             # historique de la piste
            if tid not in plate_by_tid: ocr_crops[tid] = crop    # envoie la plaque à l'OCR
            if len(tr["hist"]) >= MIN_TRACK_FRAMES:
                tr["wrong"] += 1 if is_wrong_way(tr["hist"]) else -1 (min 0)
                if tr["wrong"] >= CONFIRM_EVALS and tr["pending"] is None: tr["pending"] = time.time()
            if tr["pending"] and not tr["reported"]:
                plate = plate_by_tid.get(tid)
                if plate or time.time() - tr["pending"] > PLATE_WAIT:
                    register_infraction(tid, plate or "INCONNUE", annotated); tr["reported"] = True
        # toutes les 100 images : purge des pistes inactives (> 150 images)
        # 8 fois/s : redimensionne, JPEG, base64 -> publish ANPR/camera/frame
        # publie ANPR/vehicle/detection (classes + confiance)
        # cv2.imshow(...) ; touche « q » pour quitter
    else:
        time.sleep(0.1)
```

Machine à états d'une piste :

État	Condition d'entrée	Action
Observation	Piste créée par ByteTrack	On accumule jusqu'à 20 positions ; l'OCR tourne en arrière-plan.
Évaluation	≥ 10 positions (MIN_TRACK_FRAMES)	À chaque image : wrong +1 si sens interdit, sinon −1 (min 0). Le compteur sert de filtre anti-bruit.
Suspect (pending)	wrong ≥ 8 (CONFIRM_EVALS)	Horodatage de suspicion ; texte rouge « SENS INTERDIT » sur l'image ; OCR prioritaire.
Enregistré (reported)	Plaque validée **ou** 10 s écoulées (PLATE_WAIT)	register_infraction ; plaque = lue ou « INCONNUE » ; plus d'évaluation pour cette piste.

À la sortie (finally) : arrêt du thread OCR (running=False), libération de la caméra, fermeture de la base et déconnexion MQTT.

5. De la détection à l'affichage : la chaîne complète



5.1 Déroulé étape par étape

1. Démarrage

L'utilisateur clique sur « Démarrer » dans le dashboard. Le ui_template appelle `scope.send({cmd:"cam", payload:"ON"})` ; le nœud **router commandes** envoie le message sur sa 2e sortie vers **mqtt out ANPR/camera/control**. Python reçoit « ON » dans `on_message`.

2. Capture et détection

À chaque image, YOLO détecte la plaque et ByteTrack lui attribue un identifiant. L'image annotée est envoyée en base64 sur `ANPR/camera/frame` (8 images/s) : on voit la vidéo en direct sur le dashboard.

3. Lecture de la plaque

Le recadrage est envoyé au thread OCR (prétraitement → EasyOCR → parse_plate → vote). La plaque validée est stockée dans `plate_by_tid`.

4. Règle sens interdit

L'aire de la boîte grossit de ≥ 25 % sur 20 images, de façon répétée (8 évaluations) → piste « suspecte ».

5. Passage par la base de données

register_infraction : anti-doublon (60 s) → image preuve → **INSERT** dans infractions.db (statut EN_ATTENTE) → recherche du propriétaire dans vehicles_db.csv (`owner_info`) → e-mail.

6. Publication

Le JSON `{id, horodatage, plaque, type, statut, proprietaire:{nom, prenom, email, telephone, carte_grise}, mise_a_jour}` part sur `ANPR/infraction` (QoS 1).

7. Réception Node-RED

Le nœud **mqtt in « Infraction reçue »** a 3 sorties câblées : *extraire infos*, *préparer ligne CSV* et *agréger état*.

8. Affichage

agréger état convertit le JSON en objet `st.inf` (plaque, type, date, propriétaire, tél, carte grise, confiance) et incrémente `st.infSeq`. Le template « ANPR UI » détecte ce nouveau numéro et met à jour la fiche « Dernière infraction », l'historique, le badge « NOUVEAU », les compteurs, le journal et les bulles Statistics.

5.2 Exemple de message ANPR/infraction

```
{
  "id": 12,
  "horodatage": "2026-10-21 16:57:32",
  "plaque": "123 TUN 4567",
  "type": "SENS_INTERDIT",
  "statut": "EN_ATTENTE",
  "proprietaire": { "nom": "...", "prenom": "...", "email": "...",
                    "telephone": "...", "carte_grise": "..." },
  "mise_a_jour": false
}
```

Champ	Signification	Utilisation côté Node-RED
id	Clé primaire SQLite de l'infraction	Lien avec la base (non affiché)
horodatage	Date et heure (chaîne)	st.inf.date : fiche, historique, CSV
plaque	Matricule lu, ou « INCONNUE »	connue = plaque ≠ INCONNUE ; sinon affichage « PLAQUE NON LUE »
type	SENS_INTERDIT	Mis en majuscules, « _ » remplacé par un espace
statut	EN_ATTENTE (workflow de traitement)	Non exploité pour l'instant
proprietaire	Objet issu de vehicles_db.csv ({} si inconnu)	owner, carte, email, tel
mise_a_jour	true si la plaque a été corrigée après coup	Réception d'un 2e message pour la même infraction

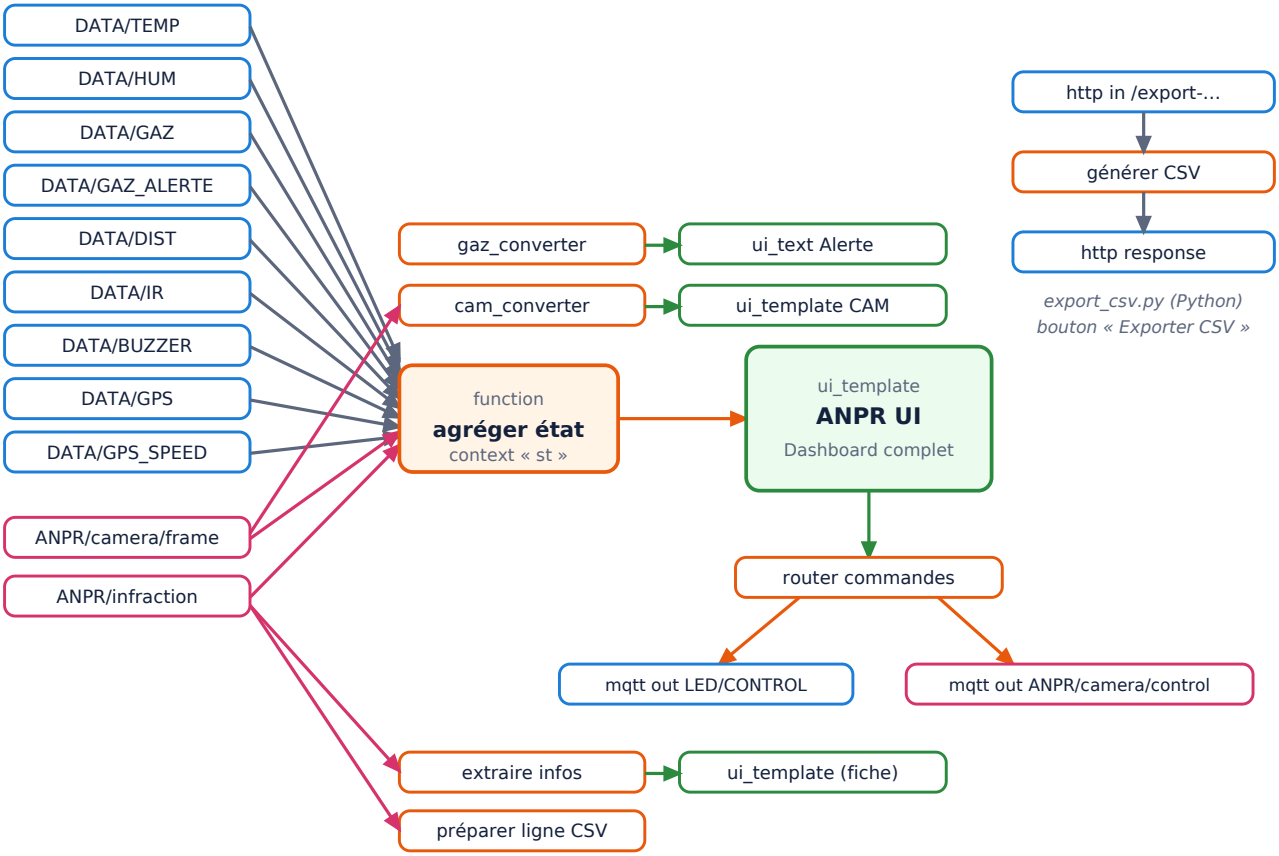
> **À retenir :** Le message ne contient pas de champ « confiance » : le dashboard affiche donc « Confiance OCR : non fournie ». Pour l'afficher, il faudrait ajouter un champ `"confiance"` dans `publish_infraction` (le code de agréger état sait déjà le lire).

5.3 Cas de la plaque non lue (« INCONNUE »)

Si après 10 s aucune plaque n'est validée, l'infraction est quand même enregistrée avec la plaque « INCONNUE » (la preuve image existe). La piste est mémorisée dans `late_update`. Si l'OCR valide la plaque plus tard, Python fait un **UPDATE** SQL, republie le JSON avec `mise_a_jour:true` et envoie l'e-mail. Le dashboard ne crée pas de doublon grâce au suivi de `infSeq` et de la date.

6. Les nœuds Node-RED (onglet « projet_iot »)

Le flow est constitué de **nœuds** reliés par des **fils** : un message (`msg`) traverse les nœuds, avec `msg.payload` (donnée) et `msg.topic` (nom du topic MQTT). Schéma simplifié des liaisons utiles au projet ANPR :



6.1 Nœuds de configuration

Nœud	Rôle
mqtt-broker (pf-bh1lt76…cedalo.cloud:1883)	Connexion partagée par tous les nœuds mqtt in / mqtt out du projet ANPR et des capteurs (même broker que le script Python). Contient host, port, identifiants.
mqtt-broker « Cedalo MES » (pf-5kwp8…)	Ancien projet de production (MES) : autre broker, non utilisé par le flow ANPR.
ui_tab « Projet ANPR », ui_group « ANPR », « CAM DATA », « DHT DATA », « GAZ DATA », « ULTRASON DATA », « Commande LED »	Organisation de l'ancien dashboard (node-red-dashboard v2) : onglet → groupes → widgets.
ui_base, ui-theme, ui-page, ui-group (Production, Maintenance, Température…)	Restes du projet MES / de la v2 de Dashboard : pages Accueil, Production, Maintenance. Non utilisés par ANPR (peuvent être supprimés).
sqlitedb	Configuration d'une base SQLite du projet MES (aucun nœud sqlite ne l'utilise dans le flow ANPR).
ui_spacer ×11	Espaces vides pour aligner les widgets.
global-config	Réglages globaux Node-RED.

6.2 Capteurs (ESP32 → dashboard)

Nœud	Type	Fonction	Sorties vers
DATA/TEMP	mqtt in	Reçoit la température	ui_gauge (Celsius 0–50) + agréger état
DATA/HUM	mqtt in	Reçoit l'humidité	ui_gauge (% 0–100) + agréger état
DATA/GAZ	mqtt in	Valeur du capteur MQ	ui_gauge (PPM 0–4095) + agréger état
DATA/GAZ_ALERTE	mqtt in	Indicateur d'alerte (0/1)	gaz_converter + agréger état
gaz_converter	function	`msg.payload = (payload == 1) ? "⚠ GAZ DÉTECTÉ" : "Normal"`	ui_text « Alérte »
DATA/DIST	mqtt in	Distance ultrason (cm)	ui_gauge (CM 0–400) + agréger état
DATA/IR, DATA/BUZZER	mqtt in	État IR et buzzer	agréger état
DATA/GPS, DATA/GPS_SPEED	mqtt in	Position « lat,lng » et vitesse	agréger état (carte Leaflet côté navigateur)
ui_button ON / OFF (LED)	ui_button	Émettent payload 1 ou 2 (ancien dashboard)	mqtt out LED/CONTROL
mqtt out LED/CONTROL	mqtt out	Publie la commande de la LED	Vers l'ESP32

6.3 Caméra et infractions

Nœud	Type	Fonction
Camera Frame	mqtt in	Abonné à `ANPR/camera/frame` : reçoit le JPEG en base64. Deux sorties : cam_converter (ancien affichage) et agréger état.
cam_converter	function	Construit une balise HTML : `<img src="…">` à partir du texte base64.
CAM	ui_template	Affiche ce HTML (`ng-bind-html="msg.payload"`) : flux vidéo de l'ancien dashboard.
ui_button Démarrer / Arrêter Caméra	ui_button	Envoient « ON » / « OFF » vers le nœud mqtt out CONTROL/CAM.
CONTROL/CAM	mqtt out	Publie sur `ANPR/camera/control` → reçu par on_message du script Python.
Infraction reçue	mqtt in	Abonné à `ANPR/infraction` (QoS 2). Trois sorties : extraire infos, préparer ligne CSV, agréger état.
extraire infos	function	Transforme le JSON en **fiche HTML** : si pas d'erreur, carte rouge « 🚫 SENS INTERDIT » + plaque + tableau (date, carte grise, propriétaire, e-mail, tél) ; si la plaque est « INCONNUE » : « Propriétaire inconnu : vérification manuelle ». S'il y a un champ *erreur* : carte orange « ⚠ ERREUR ».
(ui_template sans nom)	ui_template	Affiche la fiche HTML (styles .anpr-card) + bouton « Exporter CSV » qui télécharge `/export-infractions`.
préparer ligne CSV	function	Convertit le message en objet et **ignore** les plaques inconnues (`return null`). Le nœud n'est relié à rien et ne renvoie rien d'autre : étape préparatoire non terminée.
http in /export-infractions (GET)	http in	Point d'entrée HTTP appelé par le bouton d'export.
générer CSV	function	Exécute `python C:/projet_anpr/test/export_csv.py` avec `child_process.execSync` (module « cp » déclaré dans l'onglet Setup du nœud). La sortie texte du script est renvoyée avec les en-têtes `Content-Type: text/csv` et `Content-Disposition: attachment; filename="infractions.csv"` ; en cas d'erreur, un CSV minimal contenant le message.

Nœud	Type	Fonction
http response	http response	Renvoie la réponse HTTP (le fichier CSV) au navigateur.

À retenir : **export_csv.py** n'est pas fourni, mais d'après le nom et son usage il lit la table SQLite `infractions` et imprime des lignes CSV (horodatage, plaque, type) : c'est **le second lien entre Node-RED et la base de données**. Node-RED ne lit jamais SQLite directement : il passe par un script Python.

6.4 Le cœur : agréger état, ANPR UI et router commandes

agréger état (function)

Il reçoit les messages de **tous** les topics (mqtt in) et maintient un **objet d'état unique** `st` dans le **contexte du nœud** (`context.get/set`) : la mémoire persiste entre deux messages.

```
var st = context.get('st') || {temp:null, hum:null, gaz:null, dist:null, alerte:false, ir:false,
                               buzzer:false, img:'', inf:null, ts:0, lat:null, lng:null, ...};
var t = msg.topic, p = msg.payload;
if      (t === 'DATA/TEMP')       st.temp   = parseFloat(p);
else if (t === 'DATA/GAZ_ALERTE') st.alerte = String(p).trim() === '1';
else if (t === 'DATA/GPS')        { split(',') -> st.lat, st.lng, st.fix }
else if (t === 'ANPR/camera/frame') st.img = '' + p;
else if (t === 'ANPR/infraction') { JSON.parse -> st.inf = {...}; st.infSeq++ }
else return null;                 // topic inconnu : on ignore
st.ts = Date.now();  context.set('st', st);
return {payload: Object.assign({}, st)};   // copie de l'état complet
```

- **Pourquoi agréger ?** Le template reçoit un message unique contenant *toutes* les valeurs, au lieu de gérer 12 flux séparés.
- Pour l'infraction : construit `st.inf` = {erreur, connue, plaque, type, date, carte, owner, email, tel, conf}. connue vaut vrai si la plaque ≠ « INCONNUE » ; le propriétaire est « Inconnu » sinon.
- Le compteur **infSeq** augmente à chaque infraction : c'est le signal fiable de « nouvelle infraction » pour le dashboard.
- `Object.assign({}, st)` envoie une **copie** pour que le contexte reste intact.

ANPR UI (ui_template, portée locale)

Page complète en HTML/CSS/AngularJS. Elle observe `msg` avec `scope.$watch("msg", ...)`, copie `msg.payload` dans `scope.d` et met à jour toute l'interface. Pour envoyer une commande elle appelle `scope.send({cmd, payload})` : le message sort du nœud vers **router commandes**.

router commandes (function à 2 sorties)

```
if (msg.cmd === 'led') return [msg, null];   // sortie 1 -> mqtt out LED/CONTROL
if (msg.cmd === 'cam') return [null, msg];   // sortie 2 -> mqtt out CONTROL/CAM
return null;
```

Un nœud function peut renvoyer un **tableau** : chaque case correspond à une sortie ; `null` = rien sur cette sortie. C'est un aiguillage selon `msg.cmd`.

7. Le dashboard « ANPR UI » : fonctionnalités

Deux onglets : **Diagnostic** (valeurs brutes de chaque capteur) et **Automatique** (supervision ANPR). Un bouton bascule entre **mode sombre** et **mode clair** ; le fond est une image floutée avec des cartes en « verre dépoli » (`backdrop-filter: blur`).

Zone	Contenu et logique
En-tête	Titre, bouton thème, état « ESP32 · MQTT connecté / déconnecté ». `online()` = dernier message reçu il y a moins de 30 s (`tick − recv < 30000`).
Diagnostic : CAM DATA	Image de la caméra, plaque et propriétaire de la dernière infraction, boutons Démarrer / Arrêter, Exporter CSV.
Diagnostic : DHT / GAZ / ULTRASON	Grande valeur colorée (vert / orange / rouge par seuils), barre de niveau, graphique SVG historique (40 points), alertes, buzzer, IR, LED ON/OFF ; complétés par min / moy / max, seuils gaz, sparkline de distance, compteurs.
Journal des événements	Fil chronologique (gaz, ultrason, buzzer, IR, température, infractions) détecté sur les **fronts montants** des états.
GPS DATA	Carte **Leaflet** avec suivi de la position et trace ; **météo** de la position via l'API Open-Meteo.
Automatique : barre de statut	État de la caméra, compteurs Alertes / Infractions, heure de mise à jour.
Flux caméra en direct	Image `d.img` + plaque (autoPlate) ; badge « NOUVEAU » pendant 4 s à chaque infraction.
Fixing (incidents)	Moteur de règles qui crée des incidents INC-xxxx : flux caméra interrompu (> 15 s), matricule non lu, confiance OCR faible, MQTT hors ligne, gaz, température ≥ 30 °C, DHT sans mesure, GPS sans fix. Statuts Investigating → Fixing → Résolu.
Dernière infraction	Bandeau rouge avec le type, plaque en police monospace, horodatage, propriétaire, carte grise, téléphone.
Statistics	Bulles proportionnelles aux pourcentages : précision caméra (identifiées / lues inconnues / non lues) et alertes capteurs (gaz, ultrason, température, IR) ; la plus grande est rose.
Historique des infractions	Tableau des 100 dernières infractions de la session + export CSV (appel HTTP /export-infractions).

7.1 Logique « identifiée / lue / non lue »

Pour chaque nouvelle infraction (`infSeq` a changé) : **identifiée** = plaque lue ET propriétaire trouvé dans le CSV ; **lue, inconnue** = plaque lue mais absente du registre ; **non lue** = « INCONNUE ». Le taux de lecture = (identifiées + lues) / total.

7.2 Pourquoi un ui_template ?

Un seul nœud **ui_template** permet d'écrire sa propre interface (HTML, CSS, JavaScript) au lieu d'assembler des widgets standards. Le prix : l'interface est du code à maintenir. Le script s'exécute dans le navigateur et reçoit les messages via l'objet `scope` d'AngularJS.

8. Communication Node-RED ↔ VS Code

Node-RED et le script Python **ne se parlent jamais directement** : tout passe par le broker MQTT. Chacun est à la fois client *publieur* et *abonné*.

Direction	Topic	Côté Node-RED	Côté Python
Dashboard → Python	ANPR/camera/control	mqtt out « CONTROL/CAM » (publie ON / OFF)	`client.subscribe(TOPIC_CAMERA_CONTROL)` + `on_message`
Python → Dashboard (vidéo)	ANPR/camera/frame	mqtt in « Camera Frame » → agréger état → ANPR UI	`mqtt_client.publish(TOPIC_CAMERA_FRAME, base64…)`
Python → Dashboard (infraction)	ANPR/infraction	mqtt in « Infraction reçue » → 3 branches	`publish_infraction(...)` dans register_infraction et ocr_worker
Dashboard → Python (export)	HTTP /export-infractions	http in → générer CSV → execSync(python export_csv.py)	Script export_csv.py lit infractions.db

8.1 Lancer et tester l'ensemble

- Démarrer Node-RED et ouvrir le dashboard (ex. `http://127.0.0.1:1880/ui`).
- Dans VS Code, lancer le script Python : il affiche « □ Connecté au broker MQTT » puis « □ Système ANPR démarré - en attente de commandes… ».
- Cliquer sur **Démarrer** dans le dashboard : le terminal affiche « □ Commande reçue : ON » puis « □ Caméra démarrée » ; l'image apparaît.
- Présenter une plaque (photo ou vidéo) qui **s'approche** de la caméra : suspicion → « □ Matricule » → « □ INFRACTION #n ». La fiche apparaît sur le dashboard.
- Vérifier la base (extension SQLite de VS Code ou `sqlite3 infractions.db "SELECT * FROM infractions"`) et le dossier preuves/.

8.2 Dépannage

Symptôme	Cause probable	Solution
Dashboard « Hors ligne », valeurs « — »	Aucun message MQTT depuis 30 s : ESP32 éteint, broker injoignable, mauvais identifiants	Vérifier ESP32 et Wi-Fi, état « connected » des nœuds mqtt (point vert), identifiants du broker
Image vide « EN ATTENTE »	Caméra non démarrée ou script Python arrêté	Lancer Python puis cliquer sur Démarrer ; contrôler les logs « □ images »
Plaque toujours « INCONNUE »	Image floue, crop trop petit, confiance < 0,30, format non reconnu	Activer DEBUG_OCR, améliorer l'éclairage / la distance, ajuster OCR_MIN_CONF ou parse_plate
Pas d'infraction détectée	Trajet non « approching », pas assez d'images (MIN_TRACK_FRAMES), seuil AREA_CHANGE élevé	Tester avec un objet qui grossit nettement ; baisser CONFIRM_EVALS / AREA_CHANGE pour la démo
Infraction non répétée	Cooldown de 60 s pour la même plaque	Attendre ou réduire COOLDOWN
Pas d'e-mail	Propriétaire absent du CSV, mot de passe applicatif Gmail invalide, SMTP bloqué	Lire le log « [EMAIL] Erreur », vérifier vehicles_db.csv
Export CSV en erreur	Chemin python / export_csv.py incorrect, module child_process non déclaré	Vérifier le chemin dans « générer CSV » et l'onglet Setup (cp = child_process)

10. Fiche mémo (dernière révision)

Chaîne en une phrase

Caméra → YOLO + ByteTrack → (OCR + vote) → règle « surface qui grossit » → INSERT SQLite + propriétaire (CSV) + e-mail → MQTT ANPR/infraction → Node-RED (agréger état) → dashboard.

Chiffres à connaître

Paramètre	Valeur	Paramètre	Valeur
Images envoyées	8 / s, 480×360, JPEG 50	Confiance YOLO	0,5
Historique piste	20 positions	Min. avant évaluation	10 images
Seuil de grossissement	+25 %	Évaluations pour suspecter	8
Attente de plaque	10 s	Cooldown même plaque	60 s
Vote OCR	2 sur 7	Confiance OCR min.	0,30
Hors ligne	> 30 s sans message	Flux caméra interrompu	> 15 s
Gaz	seuils 1365 / 2730 / 4095	Température alerte	≥ 30 °C

Nœuds Node-RED à citer

- **mqtt in / mqtt out** : réception / émission MQTT ; **function** : code JavaScript (agréger état, extraire infos, cam_converter, gaz_converter, router commandes, générer CSV) ;
- **ui_template** : interface HTML/AngularJS ; **ui_gauge / ui_text / ui_button** : widgets du dashboard v2 ; **http in / http response** : API pour l'export CSV.

Fichiers du projet

Fichier / dossier	Rôle
script Python ANPR (VS Code)	Détection, OCR, base, MQTT, e-mail
models/exp-3.pt	Modèle YOLO entraîné
infractions.db	SQLite : table infractions
vehicles_db.csv	Registre propriétaires (matricule, nom, prenom, email, telephone, carte_grise)
preuves/	Images JPEG de chaque infraction (horodatage_plaque.jpg)
export_csv.py	Export des infractions en CSV (appelé par Node-RED)
flows_gps_diag.json	Flow Node-RED (import / export)

Limites : Ce guide est basé sur le code Python et le flow Node-RED fournis. Le firmware de l'ESP32 et le script export_csv.py n'étaient pas disponibles : leurs comportements sont déduits des topics et de l'usage dans le flow. Si l'énoncé de l'examen en parle, vérifier ces deux points dans vos fichiers.